Programmable Mathematics for Industrial Real-Time Control

Explicit objective functions and constraints in model predictive control

Scientific working paper | Status: research proposal | 4 November 2026

Research opportunity

Industrial controllers frequently evaluate the same mathematical model across many candidate future trajectories. The resulting objective and constraint functions are explicit and repeatedly called inside a larger optimization workflow. paceval UG proposes evaluating whether reusable mathematical graphs can support these selected computations, with particular attention to predictable latency, numerical consistency and possible future hardware execution.



Figure 1. Proposed integration boundary for a reusable mathematical evaluation kernel.

1. Existing research tools and the exact gap to investigate

acados provides high-performance optimal-control solvers and CasADi supports symbolic model construction, derivatives and code generation. The existing stack is already efficient and contains capabilities that a formula engine alone does not replace. A meaningful paceval pilot must isolate selected objective, correction or constraint functions and preserve identical derivative information and optimizer settings. [1,2]

2. Research contribution proposed by paceval

When many scenarios share a fixed formula structure, a reusable mathematical kernel could simplify the deployment and controlled modification of nonlinear costs or bounds. Additional value could arise from an independently checked formula evaluation path or a deterministic embedded pipeline. Faster arithmetic does not imply improved closed-loop control unless the complete solver and timing behavior are measured.

paceval is a mathematical engine that transforms supported text expressions into reusable computation objects. Its published C/C++ API supports preparing calculations for repeated evaluation, including batched input sets. The initial study should use local library integration. Networked REST or MCP communication would distort latency measurements for the targeted arithmetic kernel. The product documentation is linked in the references.

3. Explicit mathematical test case

Consider stage costs L_k = (x_k - x_ref)^T Q (x_k - x_ref) + u_k^T R u_k, with Q and R positive semidefinite, and objective J = sum_{k=0}^{N-1} L_k. For the illustrative scalar figure, L(x,u) = 4(x - 1)^2 + 0.3 u^2. The optimizer, constraints and required Jacobians remain with established tools unless separately validated.

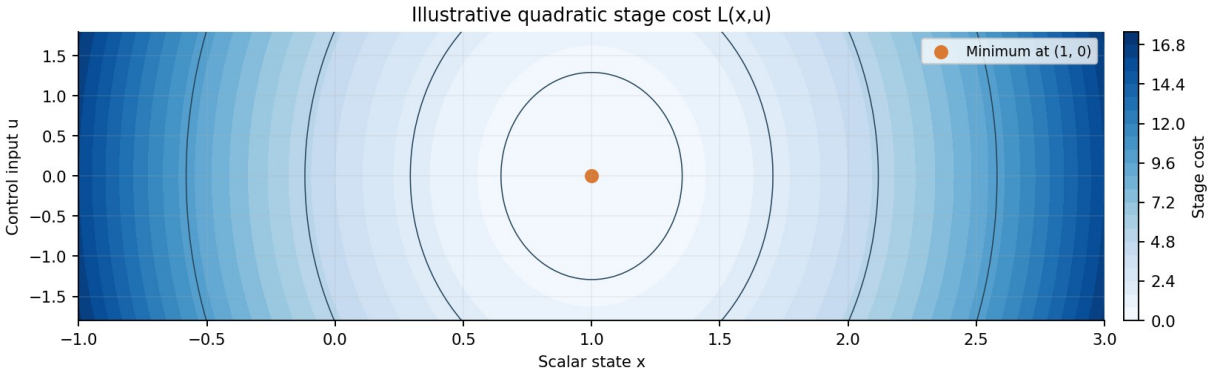


Figure 2. Illustrative quadratic stage-cost landscape. Synthetic or analytic example, not a paceval performance measurement.

The contour illustration represents a single quadratic stage cost, not an observed performance map. Its minimum for fixed u = 0 occurs at the reference state x = 1. Evaluating such functions is distinct from solving a constrained nonlinear optimal-control problem.

4. Reproducible pilot experiment

Choose a documented open nonlinear control plant. Compare native compiled C/CasADi functions with identical paceval expressions over trajectories, disturbed parameters and active-constraint cases. Keep derivative methods fixed in the optimizer. Measure graph initialization and warmed steady state separately, then evaluate the effect on solver cycles, p99 and worst observed latency, control feasibility and tracking quality.

Evaluation dimension	Study design
Mathematical correctness	Identical formulas and input domain. Independent numerical references, including boundary cases.
Kernel efficiency	Microseconds per function evaluation; median, p99 and worst observed solver latency; closed-loop objective; constraint violations; precision against a reference; allocation count and energy per decision.
Reproducibility	Thirty or more repetitions after warm-up, published inputs and software versions. Report median, dispersion and uncertainty.
Full-system validity	Include graph creation, adapters, data movement and the effect on total execution time.

5. Integration with established scientific software

Introduce an optional local C++ wrapper only around explicitly selected function evaluations. Model integration, nonlinear programming, Jacobians and Hessians remain in acados/CasADi. A real-time deployment would require known memory-allocation behavior, thread safety, timing bounds, tested failure behavior and a separate integration assessment.

6. Programmable FPGA and ASIC perspective

A future FPGA engine may process repeated candidate trajectories or scenarios through a fixed dataflow graph. Engineering evidence must cover worst-case latency, clocking, transfers, power and solver-wide effects. Safety-related deployment requires independent verification beyond these research benchmarks.

7. Expected scientific contribution

A carefully separated function-level and closed-loop comparison could establish when an additional programmable mathematical runtime is worthwhile in an already optimized embedded MPC stack.

8. Open invitation to collaborate

paceval UG invites optimal control and embedded-system research teams to design a small, open benchmark with matched cost functions, rigorous timing measurements and clearly specified extension points in the existing engine.

paceval UG is prepared to make the existing mathematical technology available for jointly defined research and to discuss targeted API extensions, open benchmark artifacts and integration formats with the scientific partners. Publication rights, licensing and intellectual property would be agreed before a joint project begins.

9. Practical work packages

First, freeze an openly specified model predictive control example. Next, implement the same explicit costs and bounds with the existing differentiation and optimization interfaces unchanged. Finally, profile closed-loop timing and evaluate a hardware path only where latency requirements justify it.

Scientific and technical sources

[1] acados official documentation and nonlinear optimal control. Open source 1

[2] CasADi documentation. Open source 2

[3] acados supported cost functions and algorithmic features. Open source 3

[4] paceval C/C++ API. Open source 4

Technical information about paceval: paceval.com

Research contact: info@paceval.com